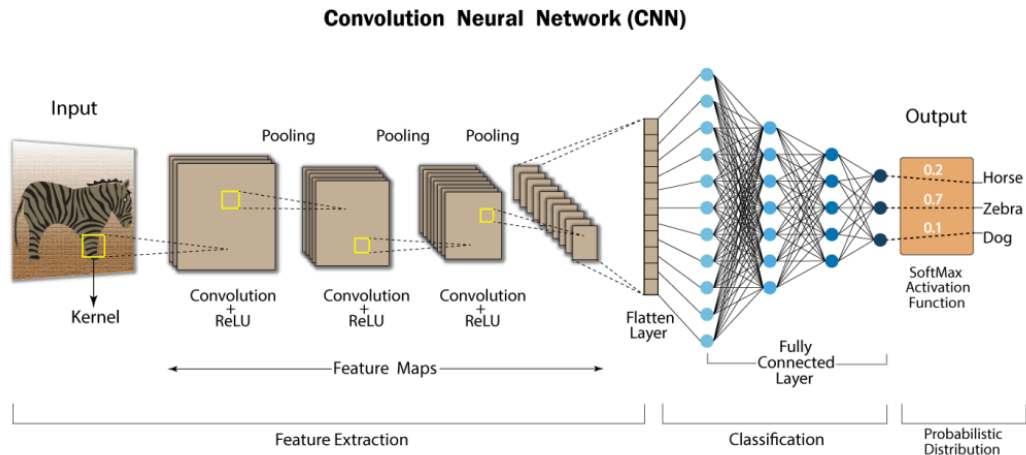


Trabajo Práctico 4: Redes Convolucionales



Introducción

En este trabajo práctico pondrá en práctica el diseño, entrenamiento y evaluación de una red neuronal convolucional (CNN por sus siglas en inglés). Deberá elegir bien los distintos elementos que componen las etapas, fundamentando sus decisiones en un informe final. Ponga en práctica todo lo aprendido anteriormente para el entrenamiento y evaluación de una red neuronal.

Utilizaremos el dataset CIFAR-10¹ que consiste de imágenes a color de 32x32 píxeles, pertenecientes a 10 clases distintas. En total se compone por 60000 ejemplos (6000 por clase), y viene separado en dos subconjuntos: 50000 imágenes de entrenamiento, y 10000 imágenes de test.

Las implementaciones de redes profundas que usaremos serán de TensorFlow con keras². Entrenar una red como las que vamos a utilizar en un Intel i7 toma una media hora de tiempo. Si tienen acceso a una GPU el tiempo se reduce por un factor 10 al menos. Pueden intentar usar Google Colab, que suele dar acceso gratuito a las GPUs de google por unas horas al día.

Deberá entregar un notebook con código, figuras y comentarios adecuados.

Ejercicio 1. Visión por Computadora

Entrene una red convolucional como la mostrada en el tutorial anterior sobre el dataset CIFAR-10. La red tiene que tener las siguientes capas:

¹ Disponible en su página original: <https://www.cs.toronto.edu/~kriz/cifar.html>, mediante la plataforma HuggingFace: <https://huggingface.co/datasets/uoft-cs/cifar10>, o en Kaggle: <https://www.kaggle.com/datasets/pankrzysiu/cifar10-python>

² Puede encontrar instrucciones para instalarlo en: <https://www.tensorflow.org/install?hl=es-419> y encontrará ejemplos para comenzar en <https://www.tensorflow.org/tutorials/keras/classification> y <https://www.tensorflow.org/tutorials/images/cnn>

Trabajo Práctico 4: Redes Convolucionales

1. Conv, filtro 3x3, 32 filtros, con padding para que conserve el tamaño de 32x32, Relu.
2. Maxpooling 2x2 (es el valor por defecto)
3. Conv, filtro 3x3, 64 filtros, con padding, Relu.
4. Conv, filtro 3x3, 64 filtros, con padding, Relu.
5. Maxpooling 2x2
6. Conv, filtro 3x3, 64 filtros, con padding, Relu.
7. Conv, filtro 3x3, 64 filtros, con padding, Relu.
8. Densa, 64
9. Densa, 128
10. Densa, 128
11. Densa, 10 (salida)

Usen como base el ejemplo del tutorial (ver la capa de aplanado al pasar de convolución a densa!), con los mismos parámetros para el entrenamiento. Obtengan las curvas de error en train y validación, y el valor de error en test del modelo final. Analice los resultados.

Plantee una modificación de la arquitectura de la CNN, utilizando sólo capas convolucionales, de pooling o densas, de forma adecuada, para obtener una mejora en el error obtenido (al menos un 3% de mejora) sobre los resultados anteriores. Trate de justificar por qué su diseño mejora respecto a la arquitectura anterior.

Ejercicio 2. Dropout

Agregue regularización por Dropout a la red planteada en el ejercicio anterior. Primero con una capa Dropout luego de la última capa convolucional, y luego agregando al anterior una segunda capa de dropout luego de la primera capa densa.

Pruebe dos valores de p en cada caso, 0.2 y 0.5 (al agregar dropout se recomienda duplicar las épocas de entrenamiento porque el azar dificulta el aprendizaje). Obtenga las curvas de entrenamiento y validación, y el error de test. Analice los resultados.

Ejercicio 3. Data Augmentation

Siguiendo otro ejemplo de keras, definamos una capa de data augmentation así:

```
img_height = 32
img_width = 32
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal", input_shape=(img_height, img_width, 3)),
    layers.RandomRotation(0.1),
```

Trabajo Práctico 4: Redes Convolucionales

```
layers.RandomZoom(0.1),  
])
```

Agréguenla como primer capa del modelo anterior (con el dropout óptimo). Reentrenen el modelo, con las curvas y el error de test (de nuevo, duplicando las épocas del original).

¿Hay mejoras? ¿Menos sobreajuste? Analice los resultados.

Ejercicio 4. Transfer Learning (Opcional, 2.5p)

Tome la red propuesta en un principio, quite todos los ejemplos de dos clases cualesquiera del CIFAR-10 para quedar con 8 clases. Tomando este nuevo dataset entrene la red por completo como antes – recuerde que ahora la capa densa final debe tener un tamaño de 8.

Tomando esta red entrenada para diferenciar entre 8 clases, queremos diferenciar ahora los ejemplos de las dos clases restantes, y sólo éstas dos.

Primero intentaremos congelando todas las capas menos la última, de la siguiente forma:

```
model_cifar8.pop()  
for layer in model_cifar8.layers:  
    layer.trainable = False
```

y reemplazaremos la capa densa final por una de 2 clases (lo siguiente vale sólo para modelos secuenciales de Keras):

```
model_cifar2 = models.Sequential([  
    model_cifar8,  
    layers.Dense(2, activation='softmax')  
])
```

Recuerde que siempre que modifica un modelo debe recompilarlo: `model.compile(...)`.

Entrene por un número adecuado de épocas. Analice los resultados.

Luego repita lo anterior pero descongelando más capas hacia atrás.

Puede usar `model.summary(show_trainable=True)` para mostrar las capas entrenables.

¿Cuántas capas necesita descongelar para lograr resultados aceptables? ¿Es conveniente hacer esto? Analice los resultados.